

Họ và Tên: Dương Hồng Sơn

MSSV: 2312953

ASSIGNMENT 2: NEAREST NEIGHBOR COMMUNICATION WITH BROADCAST

1. Diễn đạt thuật toán

Chương trình được xây dựng bằng cách chỉnh sửa `example nullnet-broadcast` của Contiki-NG. Trong chương trình gốc, mỗi node định kỳ broadcast một biến `count`, còn hàm `input_callback()` nhận giá trị đó và địa chỉ của node gửi. Phần gửi broadcast của chương trình gốc được thực hiện định kỳ bằng `NETSTACK_NETWORK.output(NULL)`.

Trong chương trình sau khi chỉnh sửa, mỗi node thực hiện hai công việc:

- Định kỳ broadcast packet và tăng biến `count`. Giá trị `count` được xem là tổng số packet node đó đã gửi (`TX_Packets`).
- Khi nhận được packet từ một node lân cận, chương trình xác định `Node_ID` của sender, cập nhật số packet đã nhận (`RX_Packets`) và tính lại chỉ số PRR.

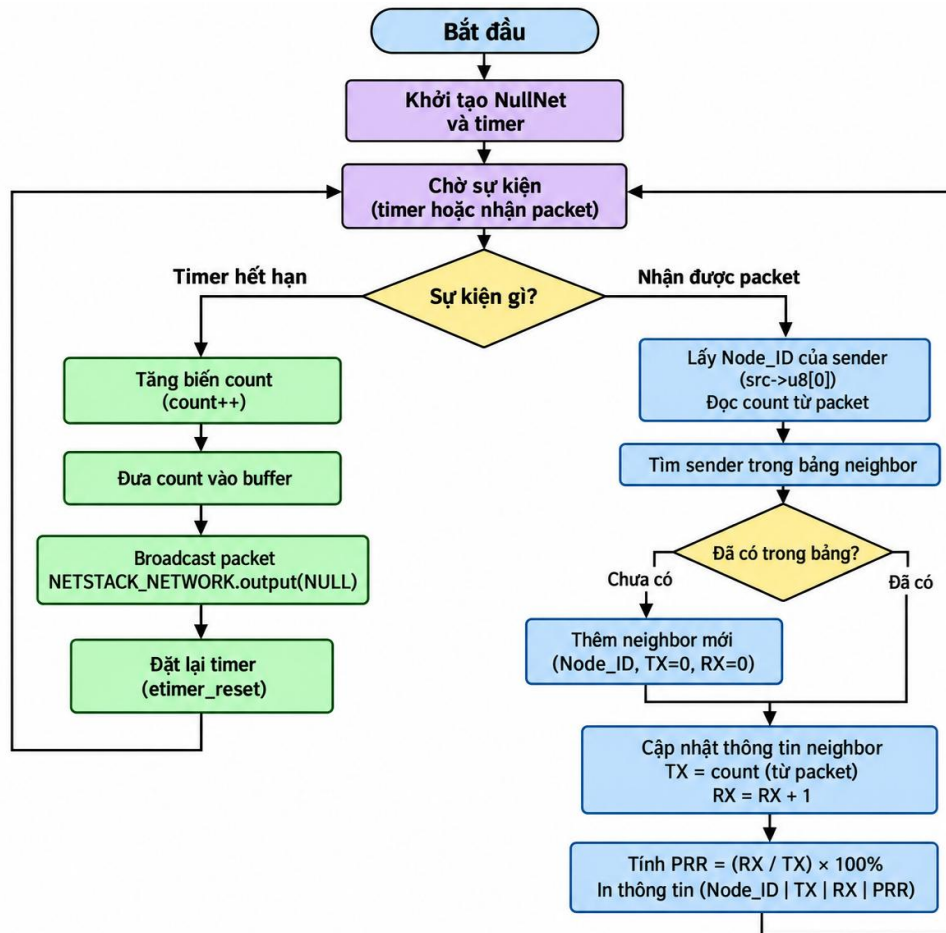
PRR được tính theo công thức:

$$PDR = \frac{RX_Packets}{TX_Packets} \times 100\%$$

Mỗi node duy trì một bảng thông tin của các node lân cận gồm:

Node_ID | TX_Packets | RX_Packets | PRR

Mỗi khi nhận được một packet mới, thông tin của neighbor tương ứng được cập nhật ngay.



Hình 1. Sơ đồ khối thuật toán thống kê packet và tính PRR của các node lân cận.

2. Mã chương trình

2.1. Bổ sung bảng lưu thông tin các node lân cận

```
#define MAX_NEIGHBORS 20

struct neighbor_info {
    unsigned node_id;
    unsigned tx_packets;
    unsigned rx_packets;
};

static struct neighbor_info neighbors[MAX_NEIGHBORS];
static int neighbor_count = 0;
```

Phần này được thêm vào để mỗi node có thể lưu riêng thông tin của từng neighbor gồm Node_ID, số packet đã gửi và số packet đã nhận.

2.2. Bổ sung hàm tìm neighbor

```
static int
find_neighbor(unsigned node_id)
```

```

{
    int i;

    for(i = 0; i < neighbor_count; i++) {
        if(neighbors[i].node_id == node_id) {
            return i;
        }
    }

    return -1;
}

```

Hàm `find_neighbor()` dùng để kiểm tra node gửi đã có trong bảng neighbor hay chưa. Nếu chưa có, chương trình sẽ thêm một phần tử mới vào bảng.

2.3. Bổ sung phần tính PRR

```

static void
print_neighbor(int index)
{
    unsigned long prr;

    if(neighbors[index].tx_packets == 0) {
        return;
    }

    prr = ((unsigned long)neighbors[index].rx_packets * 10000)
        / neighbors[index].tx_packets;

    printf("Node %u | TX=%u | RX=%u | PRR=%lu.%02lu%%\n",
        neighbors[index].node_id,
        neighbors[index].tx_packets,
        neighbors[index].rx_packets,
        prr / 100,
        prr % 100);
}

```

Hàm này tính PRR dựa trên số packet nhận được và số packet sender đã gửi. Phép tính nhân với 10000 được sử dụng để hiển thị PRR với hai chữ số thập phân mà không cần dùng số thực.

2.4. Chỉnh sửa hàm nhận packet

Trong example gốc, callback chỉ đọc biến `count` và in địa chỉ sender. Phần này được bổ sung để cập nhật bảng neighbor:

```

memcpy(&count, data, sizeof(count));

node_id = src->u8[0];

index = find_neighbor(node_id);

if(index == -1) {

```

```

index = neighbor_count;

neighbors[index].node_id = node_id;
neighbors[index].tx_packets = 0;
neighbors[index].rx_packets = 0;

neighbor_count++;
}

neighbors[index].tx_packets = count;
neighbors[index].rx_packets++;

print_neighbor(index);

```

Trong đó:

- `node_id = src->u8[0]` được sử dụng để xác định node gửi.
- `count` nhận được từ packet được xem là `TX_Packets`.
- Mỗi lần nhận thành công một packet thì `RX_Packets` tăng thêm 1.
- Sau đó chương trình tính và in lại PRR.

2.5. Chỉnh sửa phần gửi packet

Example gốc gửi giá trị `count` rồi mới tăng `count`. Trong chương trình Assignment 2, biến `count` được tăng trước khi gửi:

```

count++;

memcpy(nullnet_buf, &count, sizeof(count));
nullnet_len = sizeof(count);

NETSTACK_NETWORK.output(NULL);

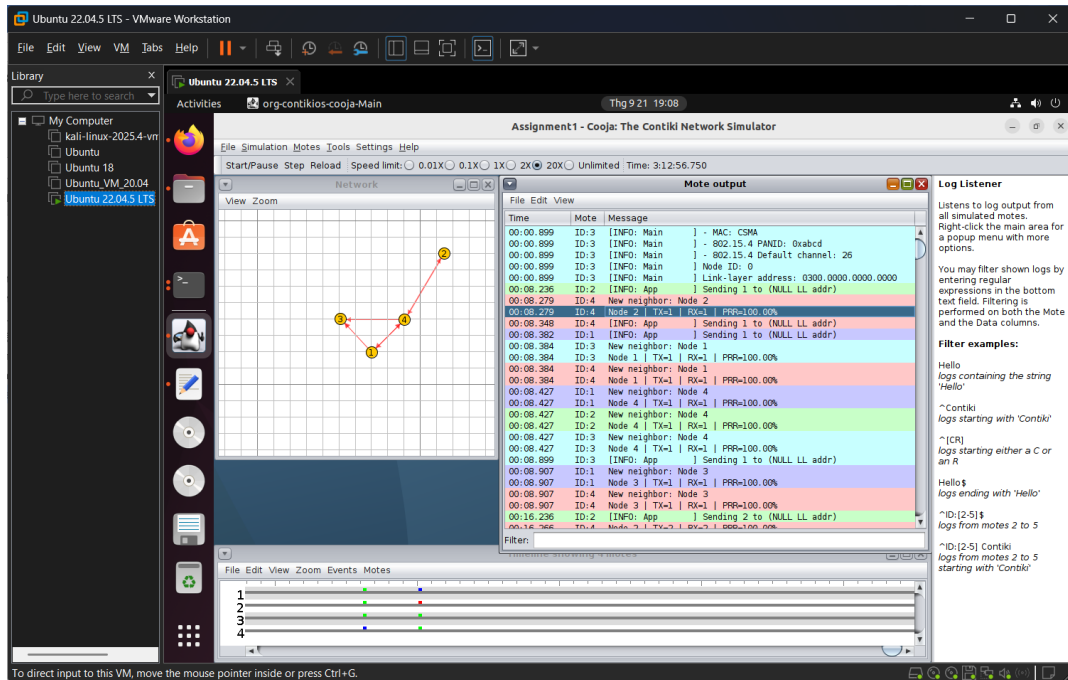
```

Việc tăng `count` trước khi gửi giúp packet đầu tiên có `TX = 1`. Thay vì `TX = 0`. Vì vậy giá trị `count` trong packet biểu diễn trực tiếp tổng số packet mà node đã gửi.

3. Kết quả mô phỏng trên Cooja

3.1. Phát hiện các node lân cận

Mô phỏng được thực hiện với 4 node sử dụng cùng chương trình nullnet-broadcast.



Hình 2. Kết quả phát hiện các node lân cận và cập nhật PRR trên Cooja.

Khi một node nhận được packet đầu tiên từ một node khác, sender được thêm vào bảng neighbor. Kết quả ban đầu cho thấy Node 2 | TX=1 | RX=1 | PRR=100.00%

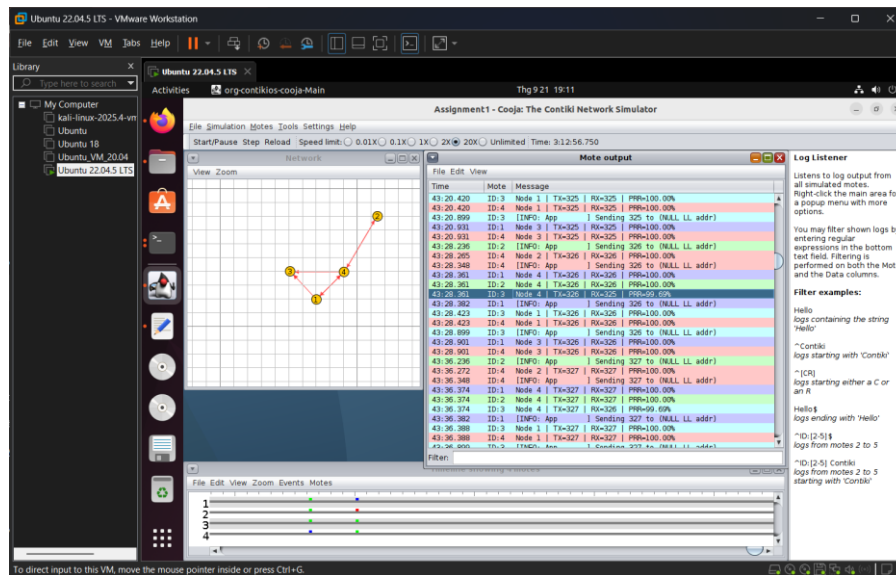
Điều này có nghĩa Node 2 đã gửi 1 packet và node đang quan sát đã nhận được đầy đủ 1 packet, do đó:

$$PDR = \frac{1}{1} \times 100\% = 100\%$$

Các node khác cũng được phát hiện và cập nhật tương tự.

3.2. Trường hợp xảy ra mất packet

Sau khi tiếp tục chạy mô phỏng, kết quả xuất hiện trường hợp: Node 4 | TX=323 | RX=322
| PRR=99.69%



Hình 3. Kết quả PRR khi xảy ra mất packet.

Trong trường hợp này, Node 4 đã gửi tổng cộng 323 packet nhưng node nhận chỉ nhận được 322 packet. Do đó:

$$PDR = \frac{322}{323} \times 100\% = 99.69\%$$

Như vậy đã có 1 packet bị mất trong quá trình truyền. Chương trình đã cập nhật đúng TX Packets, RX Packets và PRR sau khi nhận packet mới.

***Nhân xét:**

Kết quả mô phỏng cho thấy chương trình đã thực hiện được việc broadcast packet giữa các node, phát hiện các node lân cận và lưu thông tin riêng cho từng neighbor. Khi không xảy ra mất packet, PRR đạt 100%. Khi có packet bị mất, PRR giảm tương ứng với tỷ lệ packet nhận thành công.